

JDBC, REST and nested data

Pulling from databases and APIs in a notebook, and working with JSON, structs, arrays and the VARIANT type.

JDBC and REST pulls are **escape hatches**.
They are the right answer when no
connector covers your source, and the
wrong answer when one does.

Reading over JDBC

OPTION	PURPOSE
<code>url</code>	The JDBC URL, in the form <code>jdbc:subprotocol:subname</code>
<code>dbtable</code>	The table to read. Or use query for a SQL statement.
<code>user</code> and <code>password</code>	Credentials. Read these from a secret scope.
<code>partitionColumn</code>	A numeric, date or timestamp column to split on
<code>lowerBound</code> and <code>upperBound</code>	The range used to calculate the split
<code>numPartitions</code>	How many parallel reads to issue

ON THE EXAM

The same options work from SQL via `CREATE TEMPORARY VIEW ... USING JDBC OPTIONS (...)`.

Credentials

- ◆ **Never write a password into a notebook.** It ends up in Git, in the workspace, and in anyone's screen share.
- ◆ **Use a secret scope** and read the value with `dbutils.secrets.get` at runtime.
- ◆ **Secrets are redacted in notebook output**, so a printed secret shows as REDACTED rather than the value.
- ◆ **Better still, use a Unity Catalog connection** where one exists, so credentials are a governed securable.

ON THE EXAM

A hardcoded credential is a wrong answer even when the option would otherwise work. Treat it as disqualifying.

Without **partitionColumn**, Spark reads the entire table through a **single connection on one executor** — no matter what `numPartitions` says.

How the four partitioning options work together

- ◆ **partitionColumn** must be a numeric, date or timestamp column, ideally evenly distributed.
- ◆ **lowerBound** and **upperBound** define the range Spark divides into stripes.
- ◆ **numPartitions** is how many stripes, and therefore how many concurrent connections.
- ◆ **Do not set numPartitions very large.** Hundreds of concurrent connections can overwhelm the source database.

ON THE EXAM

Spark generates one WHERE clause per stripe and issues them concurrently. A skewed partition column produces skewed reads.

lowerBound and upperBound decide the **partition stride only**. They do not filter. Every row in the table is still returned.

Other JDBC tuning

- ◆ **fetchSize** controls how many rows are pulled per round trip. Defaults can be tiny.
- ◆ **Oracle's default fetchSize is 10**, which means an enormous number of round trips on a large table.
- ◆ **Use the query option** to filter and aggregate on the source, so less data crosses the network.
- ◆ **Select only the columns you need.** Wide tables are often slow for this reason alone.

ON THE EXAM

The cheapest optimisation is always reading less. Push filtering to the source before you tune parallelism.

Pulling from a REST API

- ◆ **An ordinary Python HTTP call runs on the driver only.** The executors are not involved at all.
- ◆ **Pagination, rate limits, retries and token refresh are all yours to handle.**
- ◆ **The usual pattern:** fetch pages on the driver, write the raw responses to a volume, then process them with Spark.
- ◆ **Schedule it with Lakeflow Jobs**, which gives you retries, alerting and a run history for free.

ON THE EXAM

Writing raw responses to storage first means a parsing bug can be fixed without re-calling the API. Same logic as bronze.

Where should the pulled data land?

INTO CLOUD STORAGE OR A VOLUME

Land the raw response

- Original payloads kept exactly as returned
- Re-parse after a bug without re-calling the API
- Auto Loader can then pick them up incrementally
- An extra hop and files to manage

STRAIGHT INTO A GOVERNED TABLE

Write direct

- Fewer moving parts
- Governed and queryable immediately
- Nothing to re-read if parsing was wrong
- Fine when the source can simply be re-queried

ON THE EXAM

The exam guide names both routes explicitly. The deciding question is whether the source can be asked again.

Three ways to get at a database

APPROACH	WHAT IT DOES	CHOOSE WHEN
Managed connector	Ingests a copy, handles CDC and auth	A connector exists for your source
JDBC in a notebook	You issue the query and write the result	No connector fits, or you need a specific query
Lakehouse Federation	Queries in place, no copy made	The data must not be copied

ON THE EXAM

These are not interchangeable. One copies with automation, one copies with effort, one does not copy at all.

Nested data is not a problem to be eliminated. It is a **shape**, and Spark has a type for each part of it.

The three complex types

TYPE	WHAT IT HOLDS	HOW YOU GET IN
STRUCT	Named fields of fixed, known types	Dot notation: address.city
ARRAY	An ordered list of same-typed elements	Index, or explode into rows
MAP	Key-value pairs, keys all one type	Key lookup: attributes['colour']

ON THE EXAM

A struct is one row with more columns inside it. An array is many values in one row, which is why it usually needs exploding.

Structs: dot notation

- ◆ **customer.address.city** reaches through nested structs, however deep.
- ◆ **Row count is unchanged.** You are selecting a column that happened to live inside another column.
- ◆ **Select customer.*** to promote **every field** of a struct to a top-level column in one go.
- ◆ **Field names are case-sensitive in some contexts**, notably when navigating VARIANT paths.

ON THE EXAM

If a scenario is only about reaching a nested field, dot notation is enough. No explode, no from_json.

Arrays: explode

- ◆ **explode** turns each element of an array into its own row, repeating the other columns.
- ◆ **It changes the row count.** An order with four line items becomes four rows.
- ◆ **explode_outer** keeps rows whose array is empty or null, giving them a null instead of dropping them.
- ◆ **Use size to count elements** without exploding, when you only need the count.

ON THE EXAM

Plain explode silently drops rows with an empty array. That is a quiet data-loss bug and a good distractor.

When JSON arrives as a string

- ◆ **from_json** parses a JSON string column into a struct, given a schema.
- ◆ **schema_of_json** infers that schema from a sample value, which helps you write the first version.
- ◆ **The colon operator** extracts directly from a JSON string without declaring a schema:
payload:user.name
- ◆ **from_json does not support schema evolution.** A new field needs the schema updated by hand.

ON THE EXAM

The colon operator is quick and needs no schema. `from_json` is stricter and gives you real typed columns.

VARIANT stores semi-structured data natively. Databricks now recommends it over storing JSON as strings.

Working with VARIANT

OPERATION	SYNTAX
Create from a JSON string	<code>parse_json(json_string_column)</code>
Query a field	<code>column_name:field_name</code>
Query a subfield	<code>column_name:field_name.subfield_name</code>
Query an array element	<code>column_name:array_name[0]</code>
Cast to a type	<code>column_name:field::INT</code>
Inspect the shape	<code>schema_of_variant(column_name)</code>

ON THE EXAM

Note the pattern: colon to enter the variant, dot to go deeper, brackets for array position. Paths are matched case-sensitively.

VARIANT limitations

- ◆ VARIANT columns cannot be clustering keys, partition columns, or Z-order keys.
- ◆ They cannot be used for comparisons, grouping, ordering or set operations.
- ◆ So extract frequently queried fields into ordinary typed columns, and keep the variant alongside for the rest.
- ◆ Iceberg v2 does not support VARIANT columns. Iceberg v3 does.

ON THE EXAM

That first limitation decides scenarios. If a column must be a clustering key, it cannot be VARIANT, whatever else recommends it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
JDBC read uses one core despite numPartitions	partitionColumn is missing; all four are needed
Bounds were set but every row came back	lowerBound and upperBound set stride, not a filter
A password appears in the notebook	Wrong answer. Use a secret scope.
One row per array element	explode, or explode_outer to keep empties
Reach a nested field	Dot notation. Nothing more is needed.
A JSON string column, no schema declared	The colon operator, or parse_json to VARIANT
The column must be a clustering key	It cannot be VARIANT

ON THE EXAM

Seven lines covering both halves of the module. The second and the last are the two most commonly missed.

An engineer reads a 90 million row table from PostgreSQL over JDBC into bronze. The read takes just over four hours. She sets numPartitions to 24 and reruns it. The runtime is unchanged.

The Spark UI shows one task in the stage, running for the whole four hours, on a cluster with 32 cores available.

What is wrong?

- ☐ A numPartitions is capped by the source database's connection limit
- ☐ B partitionColumn, lowerBound and upperBound are missing, so numPartitions has no effect
- ☐ C The cluster needs more executors before numPartitions takes effect
- ☐ D fetchSize must be increased before parallelism is possible

◆ PAUSE AND DECIDE

What is wrong?

- A

 Capped by the connection limit
A connection cap would reduce parallelism, not eliminate it. You would see several tasks, not exactly one.
- B

 partitionColumn and the bounds are missing
Correct. All four options are required together. Without a partition column Spark cannot generate ranges, so it opens a single connection.
- C

 More executors are needed
32 cores are already idle. Adding capacity to a job that uses one core changes nothing.
- D

 fetchSize must be raised first
fetchSize affects rows per round trip on a connection. It does not create additional connections.

THE SIGNAL

Exactly one task is the giveaway. A partly-working configuration gives you a few tasks; a missing partitionColumn gives you precisely one.

A team reads an orders table over JDBC. They only want orders with an order_id above 9,000,000, which is roughly the last month. They set partitionColumn to order_id, lowerBound to 9000000, upperBound to 9400000, and numPartitions to 8.

The resulting DataFrame contains all 9.4 million orders, not the 400,000 they expected. One partition is enormous.

What happened?

- ☐ A lowerBound and upperBound determine the partition stride only; they do not filter rows
- ☐ B The bounds were applied, but Spark re-read the excluded rows to rebalance the partitions
- ☐ C order_id is not evenly distributed, so the bounds were ignored
- ☐ D numPartitions must equal the number of rows divided by the bound range for filtering to apply

◆ PAUSE AND DECIDE

What happened?

- A

The bounds set stride, not a filter
Correct. The documentation is explicit: the bounds decide the partition stride only. All rows in the table are partitioned and returned.
- B

Spark re-read excluded rows to rebalance
Invents a rebalancing step. Nothing was ever excluded, so there was nothing to re-read.
- C

order_id is unevenly distributed
Distribution affects how evenly stripes fill. It does not cause 9 million extra rows to appear.
- D

numPartitions must match a ratio
Fabricated rule. No arithmetic relationship between numPartitions and the bounds turns them into a filter.

THE SIGNAL

To filter, put a WHERE clause in the query option. The bounds are about how to split, never about what to include.

A bronze table holds one row per order. Each row has an items column that is an array of structs, each with sku, quantity and price. Silver needs one row per item, carrying the order_id and order_date alongside.

About 3% of orders are cancelled before any item is added, so their items array is empty. Finance needs those cancelled orders to remain visible in silver.

Which approach is correct?

- ☐ A Select items.sku, items.quantity and items.price using dot notation
- ☐ B Use explode on items, then dot notation on the resulting struct
- ☐ C Use explode_outer on items, then dot notation on the resulting struct
- ☐ D Use from_json on items with a declared schema

◆ PAUSE AND DECIDE

Which approach is correct?

- A

 Dot notation on the array
Dot notation does not change the row count, so you would still have one row per order, now holding arrays of skus. The requirement was one row per item.
- B

 explode, then dot notation
Right shape, wrong on the second requirement. Plain explode silently drops the 3% of orders with empty arrays, and finance needs those.
- C

 explode_outer, then dot notation
Correct. It produces one row per item and keeps rows whose array is empty, giving them null item fields.
- D

 from_json with a schema
items is already an array of structs, not a JSON string. There is nothing left to parse.

THE SIGNAL

Two requirements again. 'One row per item' rules out A and D. 'Cancelled orders must remain visible' is what separates B from C.

An events table ingests JSON payloads whose shape varies widely between event types and changes often. A team stores the payload in a VARIANT column, which suits it well.

The table has grown large, and queries almost always filter on `event_type`, which currently lives only inside the payload. The team wants to use `event_type` as a clustering key.

What should they do?

- ☐ A Set the VARIANT column as the clustering key, since `event_type` lives inside it
- ☐ B Extract `event_type` into its own typed column and cluster on that, keeping the VARIANT alongside
- ☐ C Convert the whole payload to a struct so it can be clustered
- ☐ D Store the payload as a JSON string instead, which supports clustering

◆ PAUSE AND DECIDE

What should they do?

A Cluster on the VARIANT column

Not permitted. VARIANT columns cannot be clustering keys, partition columns or Z-order keys.

C Convert everything to a struct

Throws away the flexibility that made VARIANT the right choice. A struct needs a declared schema, and the shape changes often.

B Extract event_type to a typed column, keep the VARIANT

Correct. This is Databricks' own recommendation: pull frequently queried fields into ordinary columns and keep the variant for the rest.

D Store as a JSON string

A JSON string is what VARIANT was recommended over. It also does not make the nested field clusterable.

THE SIGNAL

The hybrid is the intended design, not a compromise. Typed columns for what you query, VARIANT for what you might.

When you pull data yourself, **you own everything the connector would have done**. When data arrives nested, name the shape first and the operation follows.